

Object Oriented Programming

Assignment 1



Submitted by:

Maham Shahzad
SP26-BAI-027

Submitted to:

Sir Shahid Bhatti

Department of Computer Science
COMSATS University Islamabad, Lahore Campus
24 September 2026

Task 1

Stakeholders and Requirement Analysis

Stakeholders and what they need most:

Stakeholder	What they need from the system
Student	A fair, quick way to book classrooms, labs, seminar rooms, or project spaces for studying, group work, or society activities.
Faculty member	Guaranteed priority for their scheduled classes, plus a fast way to request space for urgent academic needs (like a makeup class).
Administrator	A system-wide view of how spaces are being used, with reports that help plan capacity and renovation budgets.
Security staff	A clear, trustworthy record of who accessed which space and when, to help investigate incidents.
Maintenance staff	The power to mark a space unsafe or unusable right away, even if it already has confirmed bookings.

Real conflicts in the requirements:

Each conflict below is a genuine tension the scenario names or implies. The emergency-continuity condition assigned to this submission (last digit 7) is added as a fourth conflict specific to that condition, since it has to be carried through every task.

Conflict	Where the tension comes from	Decision we made before designing
Fairness vs. priority	Students expect an equal shot at any open slot, but faculty need guaranteed slots for scheduled teaching and sometimes need to bump lower-priority bookings for urgent academic work.	We set a fixed priority order: faculty-scheduled class > faculty urgent request > academic event > group meeting > individual activity. A minimum number of slots each day is also kept aside for students and can't be taken away, to preserve some fairness.
Utilization vs. safety	Admin wants rooms booked as much as possible, but maintenance staff need to be able to pull a room out of service instantly, which hurts utilization for that period.	A maintenance block always overrides any confirmed booking for that space, no exceptions. The system tries to find displaced users a new room rather than just cancelling them.
Traceability vs. privacy	Security wants booking records linked to identity and purpose, but ordinary users expect their personal details and booking reasons to stay private from other users.	Only Security and Administrator roles can see identity-linked booking details. A regular user checking a room only ever sees Available or Occupied never who booked it or why.
Emergency speed vs. normal notice policy (assigned condition)	A building closure has to cancel or move many bookings quickly, but the usual policy expects advance notice before a confirmed booking is disturbed.	Emergency closures are treated as an explicit exception that skips the normal notice period. Instead, the system guarantees a best-effort, immediate notification and keeps a permanent audit record of every booking it disturbs.

Facts versus assumptions:

Facts the scenario states directly:

- Six stakeholder groups are named: students, faculty, administrators, security staff, maintenance staff, and disability support staff.
- There are four kinds of activities individual, group meeting, class, and academic event — each with its own rules for capacity, equipment, and approval.
- A person can hold more than one role, and roles and permissions can change during the semester.
- Emergencies can cancel or relocate confirmed bookings, and the system has to keep enough information to audit this and notify the people affected.

Assumptions we made for this submission, each following directly from the gaps above:

- A fixed priority order for activity types (shown in the conflicts table) was adopted so the approval logic could actually be designed.
- A maintenance block always overrides an existing confirmed booking, no exceptions.
- Only Security and Administrator roles can see identity-linked booking details; everyone else only sees Available / Occupied.
- Emergency closures skip the normal advance-notice period but always trigger a notification attempt and an audit record.

Task 2

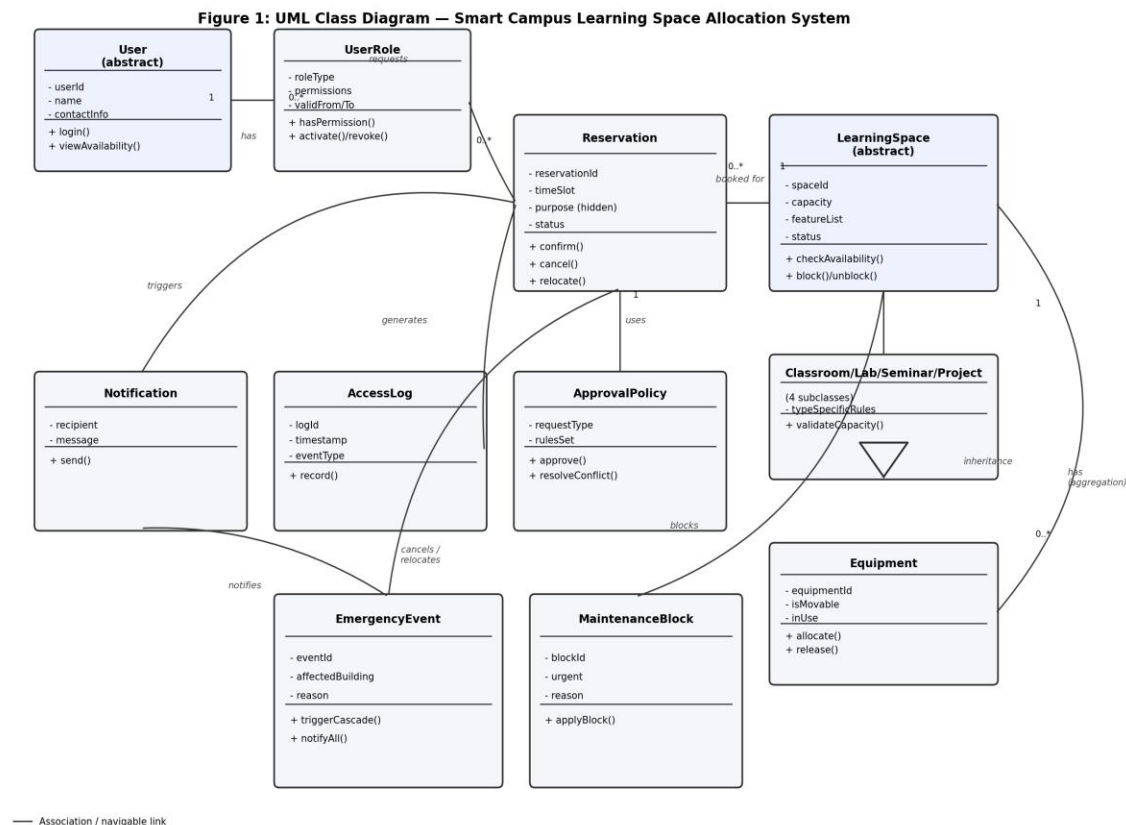
Object Oriented Conceptual Model

Ten classes are proposed below, each with its responsibility and typical attributes/operations. The model keeps the emergency-continuity condition clearly visible through its own EmergencyEvent, NotificationService, and AuditRecord classes, rather than burying that behaviour inside Reservation or LearningSpace.

Class	Responsibility	Key attributes / operations
UserAccount	Represents a person the system knows. Owns their identity and whatever roles they currently hold.	userId, name (private), contactInfo (private); authenticate(), requestReservation(), addRole()
Role (+6 role types)	Represents one set of permissions a user holds (Student, Faculty, Administrator, SecurityStaff, MaintenanceStaff, DisabilitySupportStaff). A user can hold several at once.	roleType, permissions; hasPermission(action)
Building	Groups the learning spaces inside one physical building. This is the unit an emergency closure acts on.	buildingId, name; listSpaces(), closeForEmergency()
LearningSpace (+4 subtypes)	Represents one bookable physical space and its own current status. The subtypes — Classroom, Laboratory, SeminarRoom, ProjectSpace — differ in capacity, equipment, and approval rules.	spaceId, capacity, status, isAccessible; checkAvailability(), block(), unblock()
ReservationRequest	Represents a request for a space that hasn't been approved yet — it captures what was asked for before any decision is made.	requestId, activityType, requestedSlot; submit()
ApprovalPolicy	Holds the approval rule for one activity type or situation, kept outside the Reservation class (this is the Strategy pattern).	policyName; evaluate(request)

Class	Responsibility	Key attributes / operations
Reservation	Owns the life-cycle of one confirmed-or-pending booking and enforces which state changes are allowed.	reservationId, state, startTime, endTime; approve(), cancel(reason), relocate(newSpace)
EmergencyEvent	Coordinates the ripple effect of a building closure across every reservation it touches.	eventId, cause, startTime; triggerClosure(), cancelAffected(), relocateAffected()
NotificationService	Sends and logs the notifications owed to anyone affected by a cancellation, relocation, or block.	queue; notify(user, message), logDelivery()

UML class diagram:



How the responsibilities are split up:

No single class runs the whole system. UserAccount only manages identity and the roles a person currently holds it never decides whether a booking is allowed. LearningSpace only manages its own physical status (available, occupied, blocked) and never decides who gets priority. Reservation only manages the life-cycle of one booking; it hands the actual approval decision off to a separate ApprovalPolicy object an example of the Strategy pattern covered further in Task 6 instead of hard-coding rules for every activity type inside itself. The cross-cutting work triggered by a building closure finding affected reservations, cancelling or relocating them, and notifying users is coordinated by EmergencyEvent rather than pushed into Building or LearningSpace, which keeps those classes focused on representing physical state instead of running a campus-wide response. NotificationService and AuditRecord each own exactly one job communication and record-keeping so neither Reservation nor EmergencyEvent has to manage those responsibilities itself.

Task 3

Abstraction, Encapsulation and Invariants

Three chosen classes and what each one simplifies:

LearningSpace strips a physical room down to only what the booking logic actually needs: an ID, a capacity, an accessibility flag, and a current status. Real-world detail like furniture layout or exact sensor readings is left out on purpose.

Reservation turns the real act of someone holding a space for a period of time into a single object with a controlled state, instead of spreading booking information across calendars, emails, and paper approval slips.

EmergencyEvent turns a real incident a fire alarm or a structural inspection, say into one triggering object that coordinates every cancellation, relocation, and notification that follows, instead of making staff handle each affected booking by hand.

Information that's hidden, and how it can change:

- LearningSpace hides its raw status field. It can only be read through checkAvailability(), and only changed through block() and unblock() both limited to the Maintenance staff role.
- Reservation hides its internal state field. It can only be changed through approve(), cancel(reason), and relocate(newSpace) never by direct assignment so an invalid change can't happen by accident.
- UserAccount hides name and contactInfo as private fields, only reachable through controlled access available to the account holder, Security, and Administrator roles matching the traceability-versus-privacy decision from Task 1.

Rules that must always hold true:

- A LearningSpace can never show as Available while it has a Confirmed or Active reservation overlapping that same time slot.
- A Reservation can only move along the path Requested → Approved → Confirmed → then Active → Completed, or Cancelled, or Relocated. It can never jump straight from Requested to Confirmed.
- Once a LearningSpace is Blocked by Maintenance staff, no reservation for it can reach or stay Confirmed until it's Unblocked.
- Any Reservation that's Cancelled or Relocated because of an EmergencyEvent must produce exactly one AuditRecord and at least one notification a disturbed booking can never just quietly disappear.

Because each of these fields can only change through this small set of operations, no outside class can set a LearningSpace to Available while it's Blocked, or push a Reservation straight to Confirmed without going through Approved. Encapsulation turns each rule from something a programmer has to remember everywhere into a single check performed in one place which is exactly what stops the system from ever landing in a broken state.

Task 4

Alternative Designs and Decision

The scenario says a person can hold more than one role, and that roles and permissions can change during the semester so the two natural ways to model users are compared directly against that requirement.

Criterion	Option A — Inheritance (User → Student, Faculty, ... subclasses)	Option B — Composition / role objects (UserAccount holds a set of Role objects)
Holding several roles at once	Awkward: someone who is both a Student and Faculty (like a teaching assistant) needs multiple inheritance or a special combined subclass for every combination.	Natural: UserAccount simply holds more than one Role instance in its collection, which matches the scenario directly.

Criterion	Option A — Inheritance (User → Student, Faculty, ... subclasses)	Option B — Composition / role objects (UserAccount holds a set of Role objects)
Changing permissions mid-semester	Requires replacing the object's class or migrating its state into a new subclass instance whenever a role is added or removed.	A Role object is just added to or removed from the existing UserAccount's collection at runtime; identity and history stay intact.
Duplicate behaviour / coupling	Shared behaviour common to all users (login, contact info) has to live in a base class, but role-specific data pulls toward deep, fragile hierarchies as more role combinations appear.	Shared behaviour stays in one UserAccount class; each Role stays small and independent, so new role types just mean adding a class, without touching UserAccount.

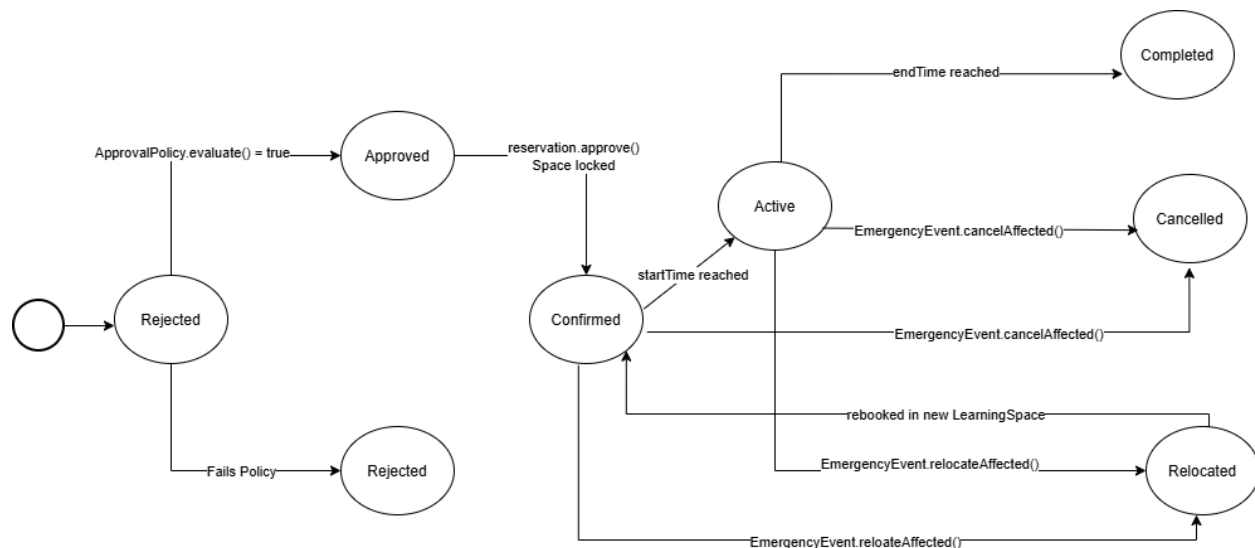
Decision:

We chose Option B composition using role objects. The scenario explicitly says a person can hold more than one role at the same time, and that this set can change mid-semester (for example, a student becoming a teaching assistant). Composition handles this naturally: Role objects are attached to or detached from an existing UserAccount without recreating the person's identity or losing their booking history. Inheritance would force an awkward class explosion for every role combination, and would require migrating a user's object identity whenever their roles change which risks breaking the very audit trail that the emergency-continuity condition depends on.

Task 5

Behaviour and Edge Cases

Reservation life cycle:



Three required edge cases:

Two simultaneous requests for the same space

LearningSpace is the single source of truth for its own time-slot state, so the check-and-lock happens inside LearningSpace itself, right when a Reservation moves from Approved to Confirmed. Whichever request gets there first locks the slot; the second request's Reservation moves to Rejected with the reason "slot no longer available" instead of being left hanging. Putting this check inside LearningSpace not inside Reservation or a central scheduler avoids a race condition and avoids turning LearningSpace's neighbours into a "god class" that has to track every other reservation in the system.

A maintenance block after confirmation

A Maintenance staff member calls `LearningSpace.block()`. `LearningSpace` doesn't decide what happens to the affected bookings beyond changing its own status – it simply exposes the confirmed reservations tied to it. Those get moved to `Cancelled`, `NotificationService.notify()` informs every affected user, and an `AuditRecord` is written with the maintenance reason. Maintenance staff and `LearningSpace` share this job because only Maintenance can trigger it, and only the space itself reliably knows which bookings currently occupy it.

An emergency cancellation or relocation

`EmergencyEvent.triggerClosure()` handles this case because it's a cross-cutting job that touches many `Reservation`, `LearningSpace`, and `UserAccount` objects at once, and shouldn't be pushed onto any single one of them. It asks `Building` for every `LearningSpace` it owns, collects every `Confirmed` or `Active` reservation against those spaces, and for each one tries `relocate(newSpace)` to an equivalent available space in the same building; if none exists, it calls `cancel(reason)` instead. `NotificationService.notify()` runs once per affected user, and an `AuditRecord` is written per affected reservation meeting the scenario's requirement to keep enough information for an audit and to notify the people affected.

Task 6

Independent Exploration: The Strategy Pattern

The Strategy pattern is a behavioural design pattern that separates an algorithm from the object that uses it. It defines a family of interchangeable algorithms behind one shared interface, so the calling object can pick or swap the algorithm it uses while the program is running, rather than having that behaviour hard-coded into it. Instead of writing one big method full of conditional branches to cover every case, the context class just holds a reference to a strategy interface and hands the decision off to whichever concrete strategy object is currently plugged in – so new behaviour can be added later as a new class, not as another branch stuffed into existing code.

In the Smart Campus Allocation System, this idea directly improves the `ApprovalPolicy` class already present in the Task 2 model. Instead of embedding approval rules for individual bookings, group meetings, classes, and academic events straight inside the `Reservation` class, each activity type gets its own concrete strategy – for example `StudentApprovalStrategy`, `FacultyPriorityStrategy`, and `EmergencyOverrideStrategy`, all implementing a shared `evaluate(request)` operation. Because the scenario deliberately leaves things like the exact meaning of fair access and limits on repeat bookings undecided, a new or revised strategy can be introduced later without touching `Reservation` at all – which keeps the approval logic easy to test on its own and open to change without shaking up the rest of the model.

Generative AI Disclosure

A generative AI tool (Claude, by Anthropic) was used to help organize, phrase, and format this assignment's analysis and to generate the illustrative UML diagram, under my direction and review. The stakeholder analysis, class design, invariants, alternative-design comparison, and behavioural analysis were reviewed and adopted by me, and I take full responsibility for every claim and design decision presented in this submission.

References

1. Association for Computing Machinery and IEEE Computer Society, *Computing Curricula 2020: Paradigms for Global Computing Education*, 2020. <https://www.acm.org/education/curricula-recommendations>
2. Seoul Accord, Section D — Graduate Attributes, especially Section D.4.1 Range of Problem Solving. <https://www.seoulaccord.org/document.php?id=79>
3. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (Strategy pattern, Chapter 5).
4. Refactoring.Guru, "Strategy Design Pattern," <https://refactoring.guru/design-patterns/strategy> (accessed September 2026).